

Facial Edge detection and segmentation

HASSAM ZAHEER BUTT
MISIS-M01087728

Contents

Abstract:	2
Introduction	2
Project Datasets and Challenges.....	2
Methodology Overview	3
Pre-processing	4
Edge Detection.....	5
First-Derivative Methods.....	5
Second-Derivative Methods (Zero-crossing based).....	6
1 st Derivative Comparison:	7
2 nd Derivative:	12
Image Segmentation.....	14
K-Means Clustering:	14
HOG + SVM:.....	15
Haar Cascade Classifier:	16
MTCNN:	16
Segmentation Results and Comparative Analysis	16
K Means:	16
HOG + SVM:.....	19
Haar Cascade:	19
Haar Cascade vs. HOG+SVM	20
MTCNN (Deep Learning Technique):	21
Conclusion	22
References:.....	23

Abstract:

Computer vision aims to give machines the ability to see and interpret visual data much like the human eye does. This report is a deep-dive analysis into how different mathematical operators and machine learning models handle the complexities of real-world facial data. I used three personal photos with different challenges: a simple baseline, an artificially blurred background, and a complex/noisy Times Square background. The project was split into two phases: edge detection and image segmentation. For edge detection, I manually implemented 1st derivative (Robert, Prewitt, Sobel) and 2nd derivative methods (Laplacian, LoG) to compare their effectiveness. For the segmentation phase, I compared the "intelligence" levels of K-Means clustering, HOG + SVM, Haar Cascades, and MTCNN deep learning. The goal is to determine which algorithms work best for facial recognition and segmentation in unpredictable, real-world environments.

Introduction

Computer vision is a field that aims to give machines the ability to "see" and interpret visual data much like the human eye does. At the core of this interpretation are two fundamental tasks: identifying boundaries (Edge Detection) and partitioning the image into meaningful regions (Segmentation). For this coursework, I am conducting a deep-dive analysis into how different mathematical operators and machine learning models handle the complexities of real-world facial data.

Project Datasets and Challenges

To truly test the robustness of these algorithms, I have selected three personal photos of myself, each presenting a unique set of challenges:

- **Image 1 (Simple Background):** This image features a clear blue sky and a uniform building colour. I chose this to establish a "baseline" for how these algorithms perform under ideal, low-noise conditions.



- **Image 2 (Blurred Background):** This photo uses a shallow depth of field. Since many modern smartphones use "Portrait Mode" to artificially blur backgrounds, I want to see how first and second-derivative operators handle these "soft" edges compared to the sharp edges of my face.



- **Image 3 (Complex/Noisy Background):** This is a colourful, high-activity photo taken in a crowded area (Times Square) that includes another person in the background. It will be interesting to see how the algorithms react to this "noise"—specifically whether they can distinguish my face from the secondary person or if they will get lost in the vibrant colours and sharp background edges.



Methodology Overview

My approach is split into two distinct phases. First, I will explore Edge Detection, which involves finding points in an image where brightness changes sharply. I will be applying and comparing multiple operators:

- First-Derivative Methods: Robert, Prewitt, and Sobel.
- Second-Derivative Methods: Laplacian and Laplacian of Gaussian (LoG).

By manually creating functions for these (without using pre-built OpenCV edge functions), I aim to understand the impact of kernel weights and convolution on the final result.

The second phase moves toward Image Segmentation. The goal here is to first find the best method to detect my face from the image and then I will use the best method in the end to segment my face from the image by using any of the edge detection algorithm. Hence, I will be comparing three different methods:

1. **K-Means Clustering:** An unsupervised approach that groups pixels by colour similarity.
2. **HOG + SVM and Haar Cascade:** A feature-based machine learning approach that looks for the specific "shape" of a face/person.
3. **Deep Learning MTCNN:** Advanced neural network techniques to achieve high-precision results.

This report will provide a comprehensive comparison to determine which algorithms work best for facial recognition and segmentation in unpredictable, real-world environments.

Pre-processing

- **Basic Operations:** Explain the transition from BGR to Grayscale. Since colour is not needed for traditional edge detection, grayscale reduces the computational load from three channels to one.
- **Gaussian Blur (The De-noising Step):**
 - **Algorithm:** Gaussian kernel uses a weighted mean where a 3x3 matrix is created that have different weights assigned to it since to blur an image we replace every single pixel with an average of itself and its immediate neighbours and the weight in the 3x3 matrix tells the computer how much importance we need to give to each pixel and neighbour. We then pass this 3x3 kernel through every pixel (from left to right, top to bottom), this process is called convolution. It calculates that weighted average math millions of times to produce the final, smoothly blurred image.
 - **Purpose:** This Algorithm is a crucial step in edge detection which decreases noise as sometimes cameras can capture tiny specks of static (noise) so if we do not blur the image slightly then the edge detection might think every piece of static is an edge hence slightly blurring an image helps with edge detection.
 - Gaussian Blur was applied to each image to remove any static before applying edge detection algorithms.
 - To show what difference this makes here is a comparison of prewitt algorithm with and without gaussian blur preprocessing.



On the left we can see there is slightly less noise and less random white dots and lines compared to the right one and even the beard and hair have much smoother edges.

Edge Detection

Let's first understand and see which edge detection algorithms I will be using. There are two different types of mathematical methods that can be used to detect edges and they are 1st and 2nd Derivative methods. The 1st derivative detects the gradient magnitude where an edge is detected when the gradient is at its peak and the 2nd derivative calculates the rate of change of the gradient where the 2nd derivative value changes from positive to either zero or negative. Since 2nd derivative measures the rate of change it is also extremely sensitive to noise.

First-Derivative Methods

- **Robert Cross Operator:**
 - **Algorithm:** Roberts uses a pair of 2x2 kernel featuring diagonally opposed 1s and -1s. It convolves this tiny sliding window across the image to calculate the 1st derivative, specifically looking for differences in pixel values along the diagonal axes.
 - **Purpose:** This algorithm is designed to compute edges extremely quickly and highlight highly localized, abrupt changes. Because the kernel is so small, it is highly precise at drawing fine edges, but this small size also makes it incredibly vulnerable to camera noise. It is best used on very clean, high-contrast, low-noise images.

- **Prewitt Operator:**

- **Algorithm:** Prewitt uses two separate 3x3 kernels containing 1s, 0s, and -1s (one to measure horizontal changes, one for vertical). These kernels are passed through every pixel using convolution to calculate the 1st derivative, which measures the "slope" or rate of change in pixel brightness. The results of the horizontal and vertical slopes are combined mathematically (using the Pythagorean theorem) to calculate the final magnitude (strength) of the edge.
- **Purpose:** This algorithm is used to detect the basic presence and direction of edges. By finding areas where the image brightness changes rapidly (a steep slope), it successfully outlines the boundaries of objects. However, because it treats all pixels in its kernel equally, it can sometimes be susceptible to noise.

- **Sobel Operator:**

- **Algorithm:** Sobel is nearly identical to Prewitt and also uses two 3x3 kernels to calculate the 1st derivative (slope) of the pixels. The key difference is that Sobel assigns a higher weight (2 and -2) to the center pixels of its kernels. It convolves these kernels across the image, calculates the gradient magnitude, and applies a threshold to draw the edges.
- **Purpose:** Like Prewitt, its purpose is to detect the presence and direction of edges, but the heavier center weights give it a slight, built-in blurring effect. This makes Sobel much better at ignoring random background noise than Prewitt, which is why it is one of the most famous and widely used basic edge detection algorithms in computer vision.

Second-Derivative Methods (Zero-crossing based)

Explain that these search for points where the second derivative changes sign.

- **Laplacian Operator:**

- **Algorithm:** Unlike the previous algorithms, Laplacian uses only one 3x3 kernel (usually with a heavily weighted negative centre, like -4, surrounded by 1s). It convolves this across the image to calculate the 2nd derivative, which measures the "rate of change of the slope" (the curvature). Instead of looking for steep slopes, it mathematically compares every pixel to all four of its neighbours simultaneously to see if it is a peak.

- **Purpose:** The purpose of the Laplacian is to find the absolute, mathematically precise centre of an edge rather than drawing a thick outline. Because it doesn't care about direction and looks everywhere at once, it is fantastic for finding the exact peak of an edge. However, this omnidirectional scanning makes it aggressively sensitive to static and noise, meaning it almost always requires a pre-blur step.
- Laplacian of Gaussian (LoG):
 - Algorithm: This is a hybrid algorithm that combines two steps into one continuous workflow. First, it applies a Gaussian Blur to the image to smooth out the noise. Second, it applies the Laplacian (2nd derivative) operator. Finally, instead of just looking for high numbers, it scans the resulting math for "Zero-Crossings"—the exact pixel where the mathematical values violently flip from positive to negative.
 - Purpose: The purpose of LoG is to get the best of both worlds: the extreme, single-pixel-thin precision of the Laplacian, without the disastrous noise sensitivity. By utilizing the Gaussian blur first and relying on Zero Crossings to pinpoint the edge centre, LoG produces highly accurate, continuous, and clean object outlines.

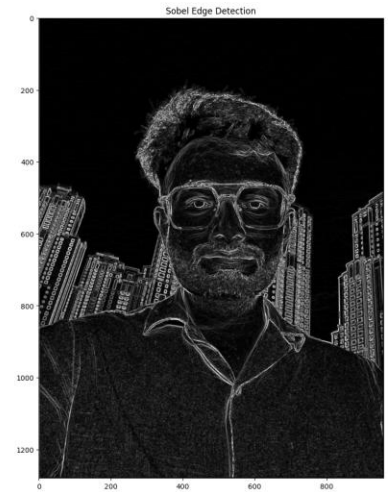
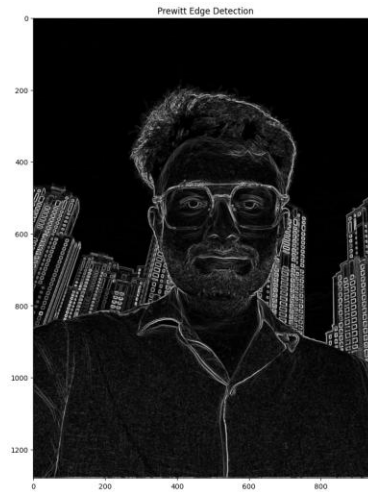
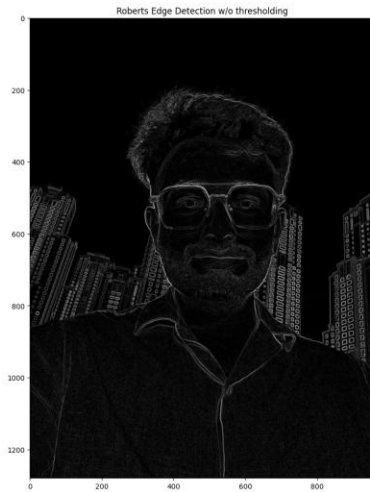
Now that we have an understanding of each method, we are going to compare these edge detection algorithms with each other and determine which algorithm works best for each image.

First, I will be comparing the 3 1st derivative methods with each other and then I will be comparing the best one of the 1st derivative methods and then compare those to the two 2nd derivative methods and see what difference do I get. I have added thresholding on the results of my edge detection algorithms to make the lines more visible.

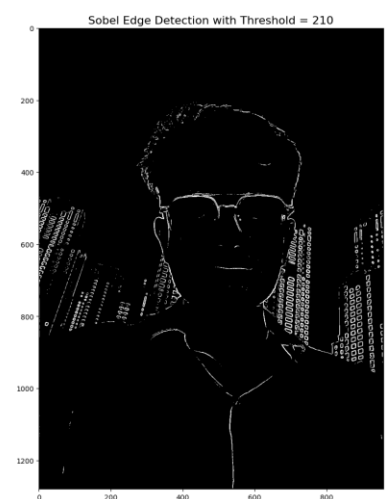
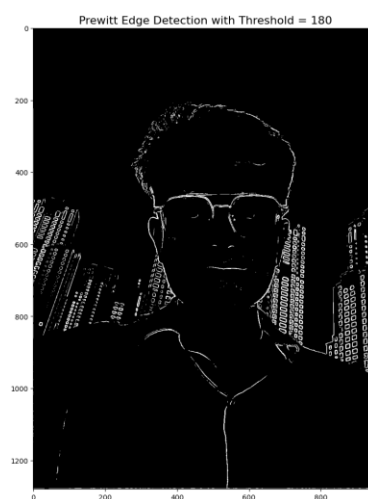
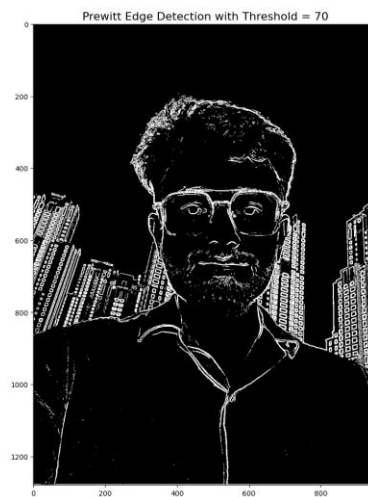
1st Derivative Comparison:

Image 1:

Without Thresholding



With Thresholding



As we can see without thresholding this image has greyish outlines so when we applied thresholding the lines are much clearer and after trial and testing

values of thresholding we can make the edges more apparent and the higher threshold I used the lighter the edges got but also removed a lot of background noise (for example the buildings in the back but it also removed my facial features while keeping the outlines visible and identifiable that yes that is a face or a silhouette of it.

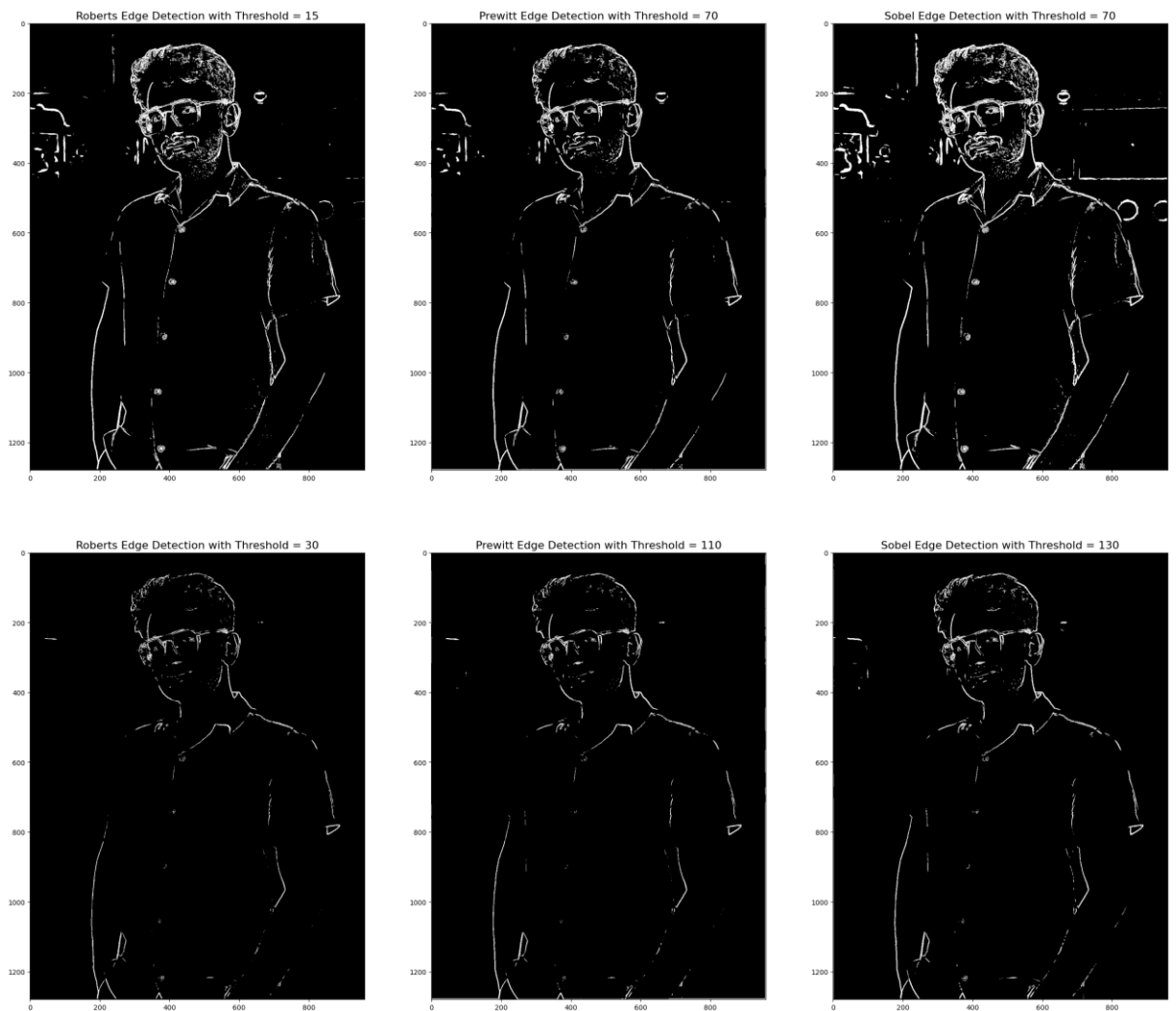
Now comparing the results for each of these methods we can see that each of these worked well and clearly defined all of the edges but we if talk in terms of edges of the faces and after playing around with the threshold I believe the Prewitt algorithm did the best job in separating the face from the buildings or it would be easiest to detect a face excluding facial features. But it mainly depends on our goal we can change our threshold accordingly.

Image 2:

Without Thresholding:



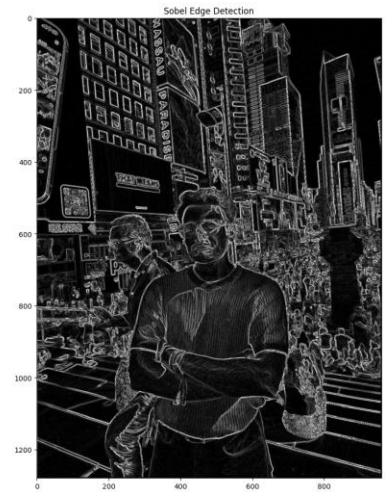
With Thresholding:



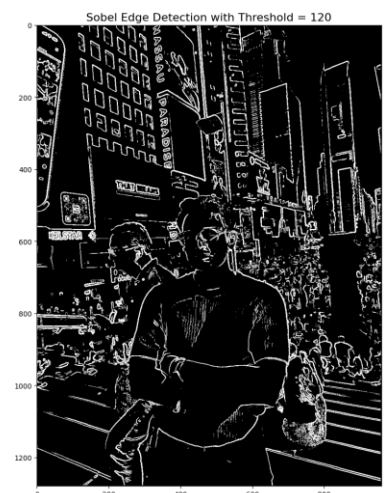
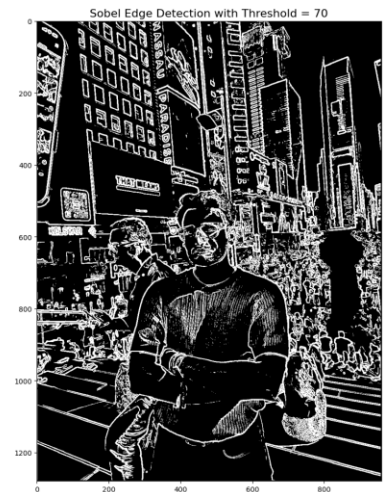
For the image that already had a blurred background, the algorithms did a great job at finding the edges. After I applied thresholding, I was able to get rid of most of the background lines while still keeping the edges of my face sharp. I found that once I hit a certain threshold, all the methods I tested were able to define my face edges well enough to be clearly recognizable.

Image 3:

Without Thresholding:



With Thresholding:



This image was by far the most complicated since it's full of people and a ton of different colours, which I knew would make edge detection really tough. Even with thresholding, I couldn't get it to look right; when I tried to go higher to clear out the background, it just made my own face look blurry and unrecognizable.

It's also worth noting that there's another person on my right whose face is also pretty clear, so I'm curious to see if the segmentation algorithms will get confused and pick them up as well.

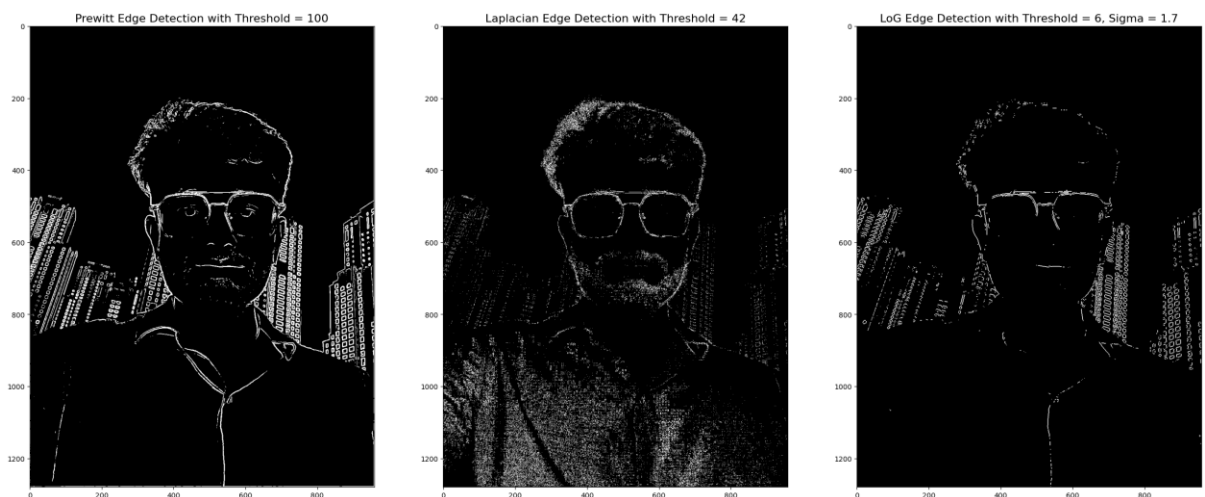
In the end, none of the algorithms were able to cleanly separate my face from the background, even with a bit of Gaussian blur to help with the static. It just shows that when an image has this many points where the colours are jumping around, the filters see everything as an edge. If I wanted to fix this, I'd probably need to manually blur the background way more to kill off that noise before running the edge detectors.

2nd Derivative:

I will be using Prewitt's results to compare with the other two and I will also be showing one preview of what seems to be the best result in terms of thresholding after manual trial and error.

Image 1:

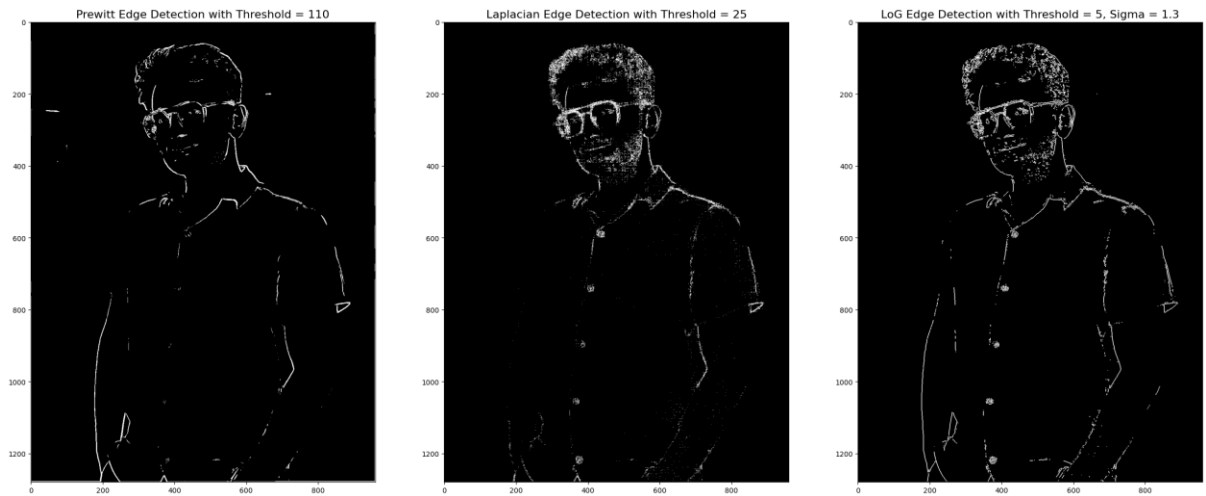
With Thresholding:



In this image Laplacian seems to give out the best result in terms of removing/hiding the background and keeping the face outline intact. Although, we can see that that the Laplacian has captured very slight changes within the shirt as well because without the gaussian blur it is very sensitive hence even a slight change in the rate of the shirt is captured by the algorithm. However, the thresholding value selected seems to have removed most of the buildings in the background making it very easy to identify my face compared to the other two. In Laplacian of gaussian the silhouette seems to be well captured but the background buildings are still there and if we compare this to Prewitt it also has a similar result but with facial features intact as well.

Image 2:

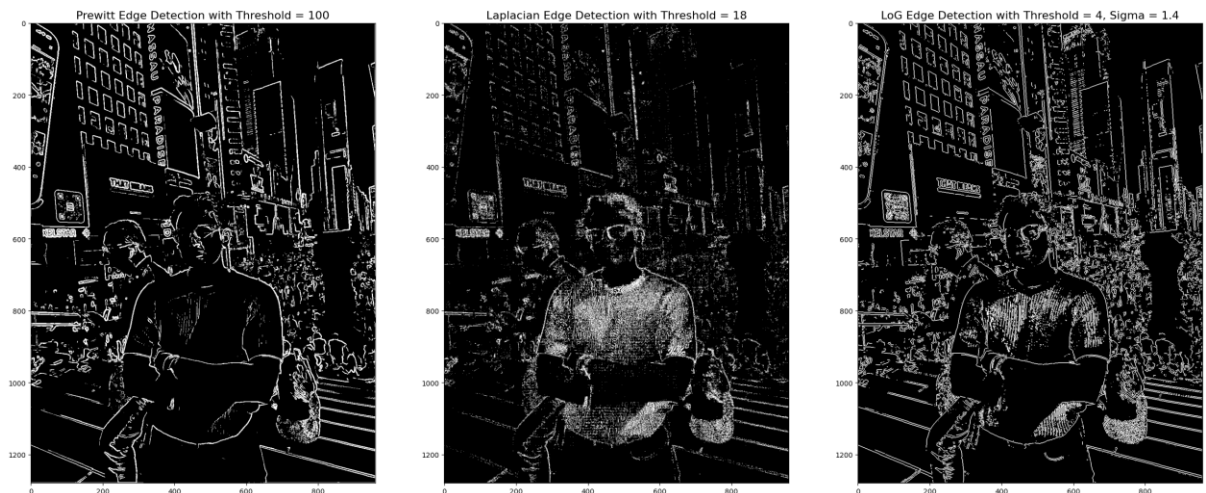
With Thresholding:



This image which already had its background blurred had the best results with all of the edge detection algorithms however there were still very slight random edges in the background but with both 2nd derivative methods there is no sign of any random background edges, and it was also able to capture the facial features really well.

Image 3:

With Thresholding:



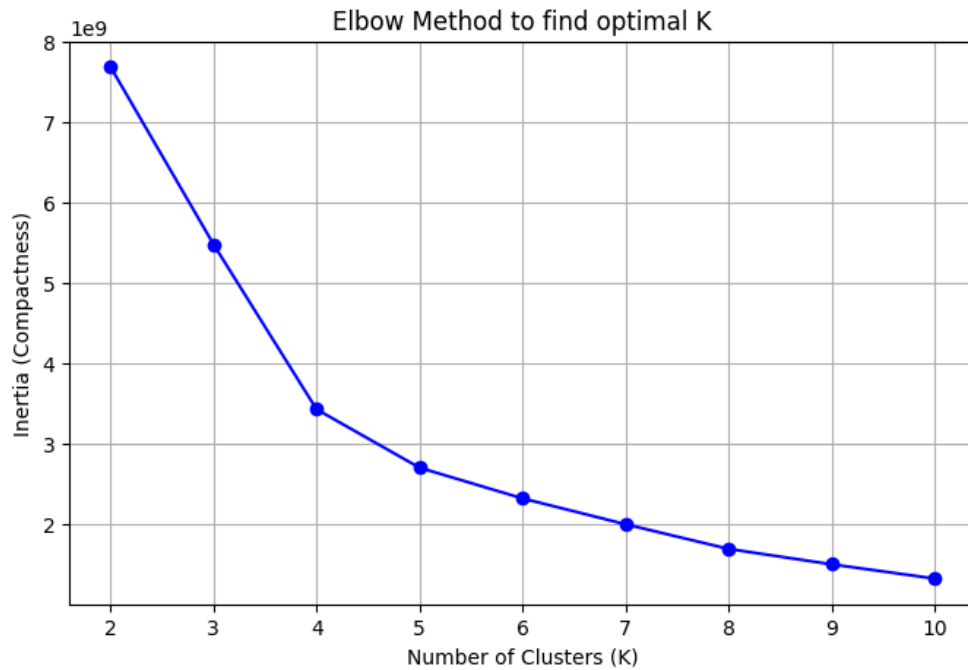
This image is the most complicated image as it has so many colour variations and the rate of change of these colours are a lot as there are a lot of people in the background wearing different colour of clothes, it detects all of them as edges so without any other image operations it would be very difficult to successfully

separate my face through edge detection algorithms without performing or using other filters.

Image Segmentation

K-Means Clustering:

- **Algorithm:** This algorithm selects K amount of random data points/pixels on the image that serve as the starting point for the centroids and then pixels are assigned a centroid based on the colour similarity using Euclidean distance to the central point and then the mean of those centroids is calculated which then becomes their new position and this keeps happening until now change or minimal change occurs in the movement of centroids.
- **Purpose:** The primary goal of the K-means clustering algorithm is to partition a dataset into K distinct, non-overlapping subgroups (clusters). In the context of computer vision and image processing, it is used for image segmentation—dividing an image into regions based on the characteristics of its pixels, such as colour or intensity. By grouping similar pixels together, the algorithm can effectively separate objects from their backgrounds or identify different textures within a scene.
- **The Elbow Method:** This method is used to find out the optimal number of centroids. This involves plotting the Inertia, the sum of squared distances of all points within a cluster from their centroid, against different values of K. The elbow of the resulting curve represents the optimal cluster value where the decrease in inertia becomes constant, providing the best balance between simplicity and detail.



HOG + SVM:

- **Algorithm:** This is a two-step process involving a feature descriptor (HOG) and a classifier (SVM).
 - **HOG Extraction:** The image is divided into small 8×8 pixel "cells." For every pixel, the gradient magnitude and orientation are calculated. A 9-bin histogram is created for each cell, representing orientations from 0 to 180 degrees.
 - **Block Normalization:** Cells are grouped into larger blocks (e.g. 2×2 cells). The histograms are normalized within these blocks to account for variations in lighting and shadows.
 - **SVM Classification:** The resulting feature vector, which represents the shape of the image, is fed into a Support Vector Machine. The SVM uses a hyperplane to mathematically separate data points that represent "Face" from those that represent Background.
- **Purpose:** Unlike K-Means, which only looks at colour, HOG focuses on the structure and shape of objects. It is specifically designed to be invariant to changes in illumination and small rotations. This makes it significantly more powerful for your "Times Square" image, as it can ignore the vibrant colours of the background signs and focus purely on the geometric gradients that define a human face.

Haar Cascade Classifier:

Algorithm: Haar Cascades works by scanning the photo using small rectangular templates called Haar features. What they're doing is calculating the difference in brightness between light and dark areas to find specific facial patterns. To keep the math super fast, the computer uses an Integral Image, which lets it do these calculations in constant time without slowing down. Since there are way too many possible features to check, it uses AdaBoost to pick out only the best ones that actually matter for a face. The "Cascade" part just means it checks these features in stages, so it can quickly throw away background parts that clearly aren't a face and only focus its power on the areas that actually look right.

Purpose: The main reason I'm using this is because it's built for high-speed, real-time detection. Methods like HOG + SVM or Deep Learning might be more accurate, but Haar Cascade is way lighter and faster. This makes it perfect for things like face tracking on a phone or live security footage where you need results in milliseconds rather than a perfect pixel-by-pixel segmentation. It's really good at ignoring all that complex background noise and pinpointing exactly where a face is almost instantly.

MTCNN:

Algorithm: MTCNN is a deep learning model that works through three different stages of convolutional networks in a chain. First, it uses the Proposal Network (P-Net), which is a shallow network that quickly scans the whole image at different sizes to find candidate windows that might have a face. Then it moves to the Refinement Network (R-Net), which is more complex and its job is to filter out most of the false positives from the first stage by looking much closer at the details. Finally, there's the Output Network (O-Net), which is the most powerful part of the system. This provides the final high-precision bounding box and also identifies five facial landmarks like the eyes, nose, and the corners of the mouth.

Purpose: The main goal of using MTCNN is to get the most accurate detection and alignment possible. Unlike Haar Cascades, which rely on rigid mathematical patterns, MTCNN uses features it has learned from seeing millions of faces. This makes it way more robust when it comes to handling things like different face angles, lighting issues, or parts of the face being covered—which were exactly the kind of challenges I ran into with Image 3.

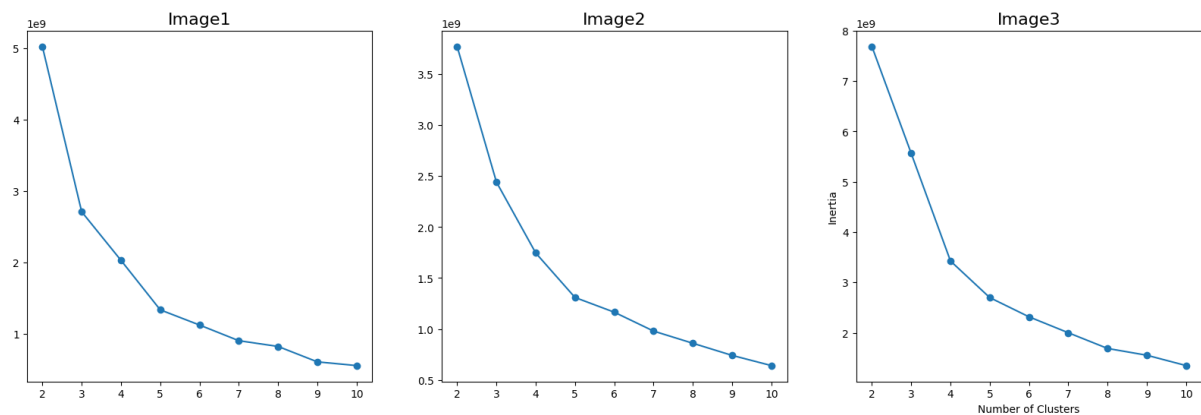
Segmentation Results and Comparative Analysis

K Means:

Before applying the K means algorithm I first applied Elbow method and plotted the inertia for each image to find the best possible value of clusters before starting K means so that I don't have to do much trial and error. But also depending on my goal I can

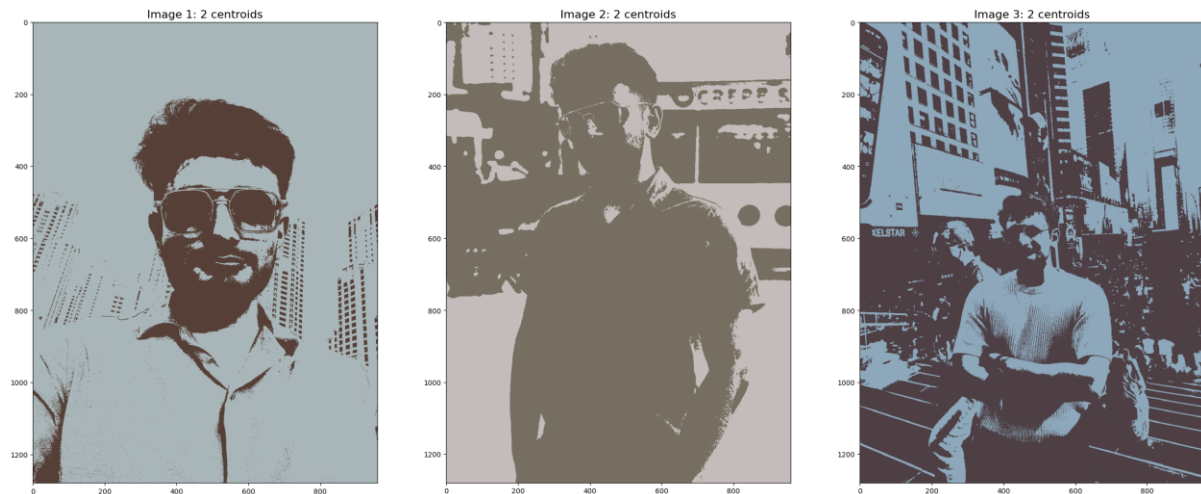
switch my values around that elbow as well, so it is much more efficient to find out the optimal K value first.

Starting with Elbow Method on all images:

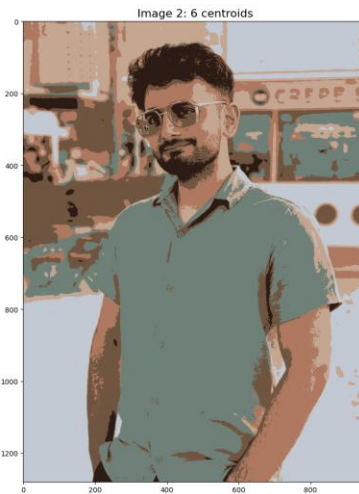
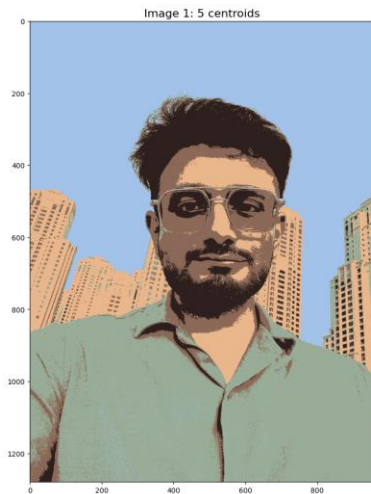


All of results seem to suggest that the best number of centroids to have is around 5 to 7. Hence, we will now use 3 different K values to show how under-fitting, over-fitting and the optimal K value would look like to see how different each result is from each other how does it affect our goal of segmenting the face from the background on all three images.

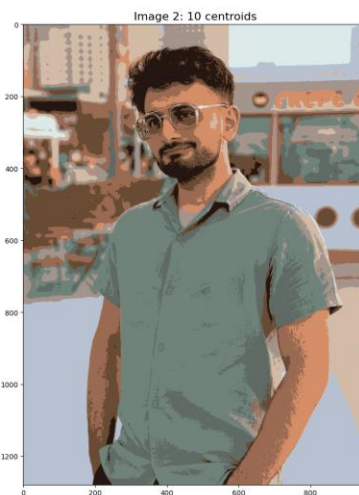
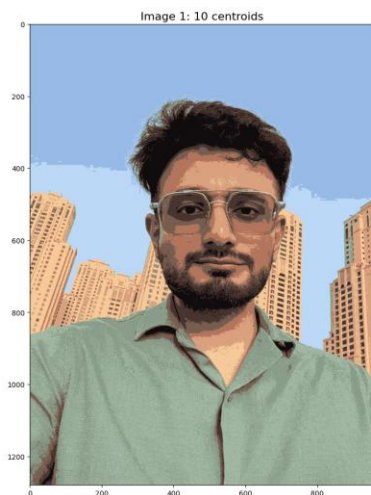
Underfitting:



Optimal:



Overfitting:



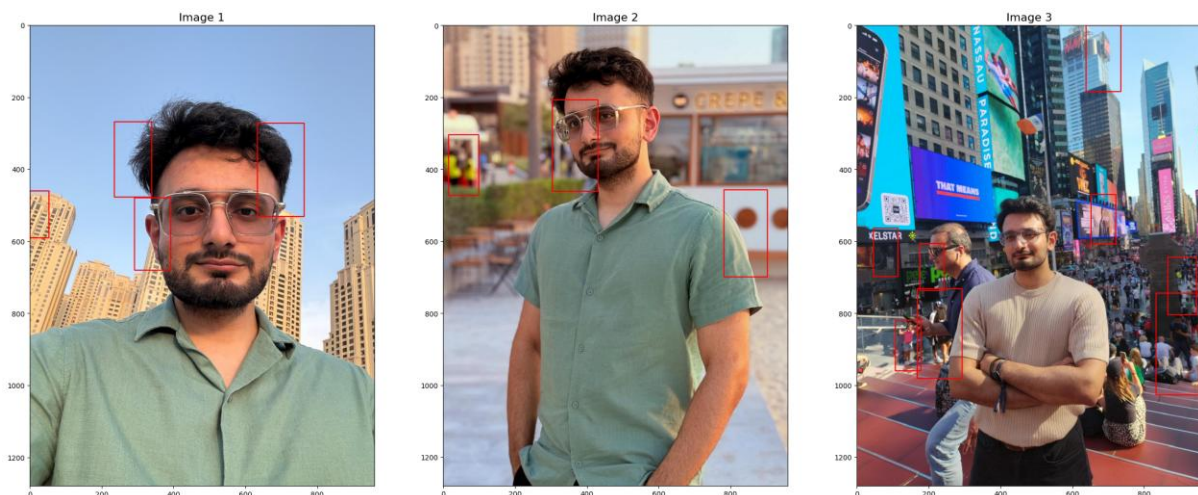
Observing image 1 since this image is comparatively simple with simple background it is safe to assume that face segmentation can occur as the edges are easier to detect with the cluster value 2 or the optimal value even though the image with 2 centroids were able to segment the face it does not work every time as more the colour differences increase the more complicated the image will become as we can see in the other two images face segmentation on those two will be very difficult or not possible as the face is unrecognizable compared to the results in the optimal section.

In the Optimal Clustering phase ($K=5$ for Image 1, $K=6$ for Images 2 and 3), the algorithm achieved the best as suggested by the elbow method. For the simple background of Image 1, the facial region was isolated as a distinct cluster because its RGB coordinates were mathematically distant from the blue-sky coordinates. However, in Image 2 which is a bit complex as some of the background and skin colour seems to be in the same cluster hence this optimal value is struggling to segment the face into a different

cluster. This highlights the inherent weakness of unsupervised colour clustering because it lacks contextual awareness of the object's geometry. And as for image 3 there isn't much difference between overfitting and the optimal value as it had so many colours that even clustering will not work properly in segmenting my face.

In terms of face segmentation, images that have colourful backgrounds and a lot of variations, k-means clustering is not the best method as it sometimes groups the colour of the background in the same centroid as the skin colour.

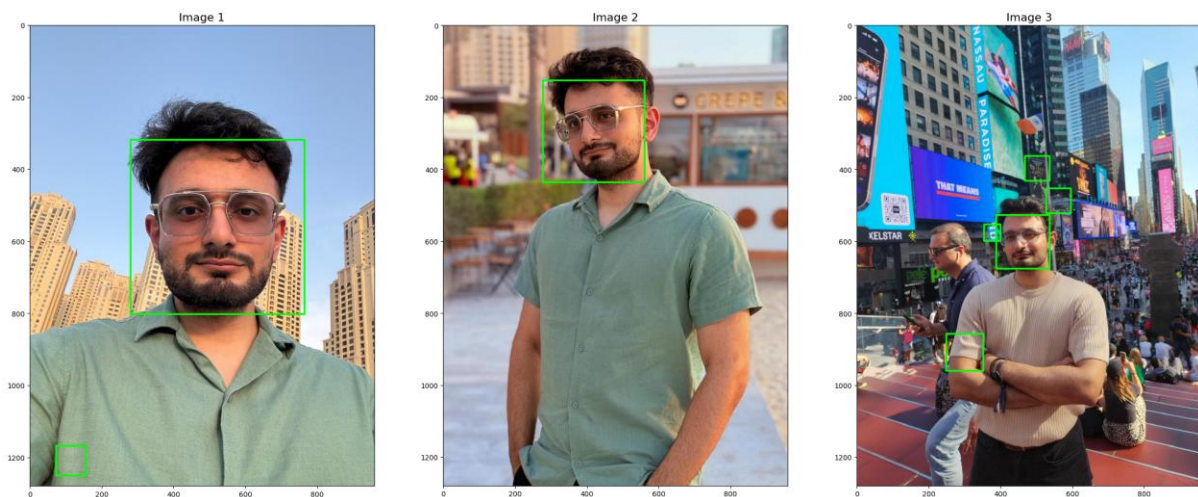
HOG + SVM:



Because I am using the `HOGDescriptor()` combined with `getDefaultPeopleDetector()`, the algorithm is struggling with my images. The main issue is that this specific detector was trained almost exclusively on full-body images of pedestrians on the street. It mathematically expects a complete silhouette: head, arms, torso, and legs. Since my legs are not visible in all three photos, the algorithm fails to recognize me as a person. This confusion is why the model draws several false positives in Images 1 and 2. Interestingly, Image 3 perfectly highlights this limitation, the algorithm successfully detects a few background people whose entire bodies are visible, but it completely misses me in the front.

Haar Cascade:

To properly detect a face cv2 offers another algorithm called Haar Cascades which is trained to detect faces rather than a whole person. Let's implement this and see how different this is from HOG+SVM implementation.



Haar Cascade vs. HOG+SVM

After running the Haar Cascade on my three test images, the results provided some interesting data for comparison:

In terms of performance and accuracy Haar Cascade was successful in locating my face in all three scenarios, which is an improvement over the colour-based K-Means clustering that often merged my face with the background. Unlike the HOG+SVM implementation which was originally designed for detecting whole people (pedestrians) the Haar Cascade is tuned for specifically detecting facial features which even includes a person wearing glasses for example. This specification allowed it to be much more confident in its detection.

Analysing the False Positives

Despite its success, the algorithm was not perfect. I observed one false positive in Image 1 and several in Image 3 (Times Square).

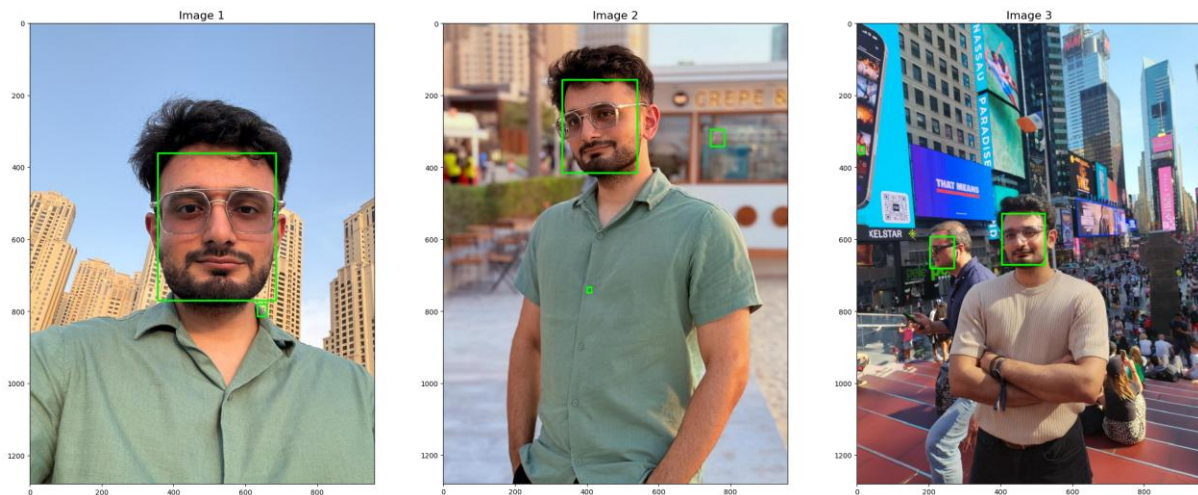
- Image 1: Even in a simple background, a false positive can occur if the shadows on a building or a window frame create a light/dark pattern that mathematically mimics the features that Haar Cascade considers as a nose, mouth etc.
- Image 3: The chaotic environment of Times Square is a nightmare for Haar Cascades. Because the background is filled with high contrast signs and other people, the algorithm found multiple regions that shared enough face like patterns to trigger a detection.

Detection vs. Segmentation

It is important to note that like HOG+SVM, Haar Cascade provides a bounding box (detection) rather than a pixel-perfect segmentation mask. While it is much faster and

more accurate at finding where I am in a crowd, it still doesn't tell the computer exactly which pixels belong to my skin and which belong to my shirt, which is where Deep Learning techniques would eventually be required for total precision.

MTCNN (Deep Learning Technique):



The MTCNN results represent the highest level of intelligence in my testing pipeline. The performance difference between this and the classical methods was clear:

Precision and Secondary Detection (Image 3)

The most impressive result was in **Image 3**, where MTCNN not only located my face but also accurately detected the person standing behind me. This demonstrates the model's **semantic understanding**; while the Haar Cascade saw the background person as noise or a false positive, MTCNN correctly classified the features as a human face. This is because deep learning models learn the hierarchical features of a face (edges -> shapes -> features) rather than relying on human-defined gradients like HOG.

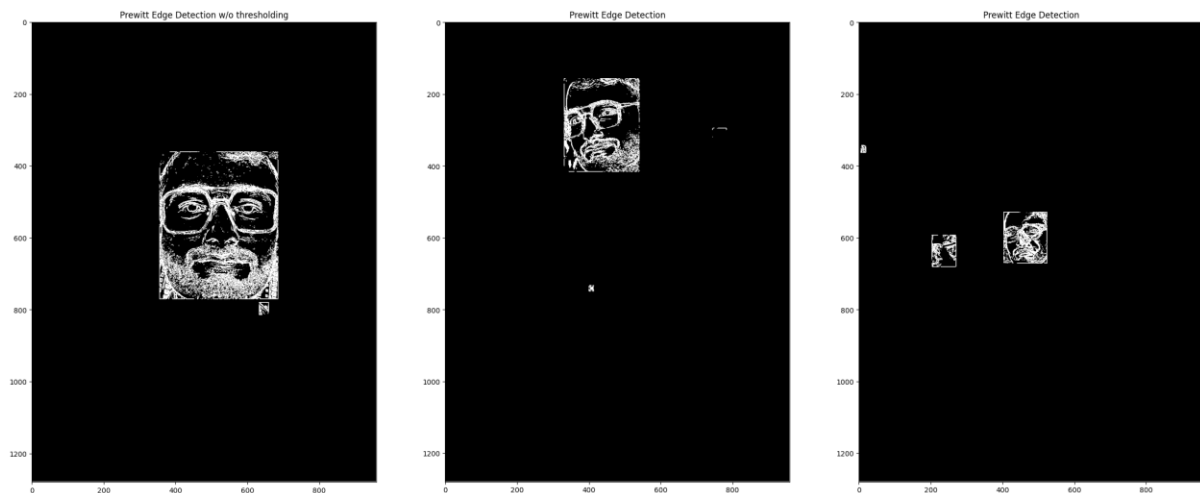
Handling False Positives (Images 1 & 2)

While MTCNN was the most accurate, I did observe minor false positives in the simple and complex backgrounds of Images 1 and 2.

- **The Technical Cause:** These usually occur when the P-Net (the first stage) identifies a texture that coincidentally shares the same feature density as a face.
- **Comparison:** However, these were much less frequent than the false positives in the **Haar Cascade** results. MTCNN's three-stage refinement process (R-Net and O-Net) acts as a rigorous filter, ensuring that only the most "face-like" regions make it to the final output.

Final Image segmentation using MTCNN and Prewitt algorithm:

Since we are able to detect the faces through MTCNN we can then perform an operation where we can blur out the whole image apart from the box that was detected as a face and then apply an edge detection algorithm in it so that we can truly segment the face using an edge detection as we can see in the result below.



Conclusion

After finishing all these tests, it's really clear that every algorithm has its own unique strengths and weaknesses depending on the environment you're using them in. My findings show that while basic edge detection is a great foundation, it's heavily affected by things like background noise and colour changes. I found that first-derivative operators like Sobel were pretty consistent, but the second-derivative methods like the Laplacian actually gave much thinner and more precise outlines, as long as I used a Gaussian blur first to kill off the static.

In terms of segmentation, there was a clear progression in "intelligence" from simple colour grouping up to true semantic understanding. K-Means clustering is efficient if you have a simple background, but it fails in complex environments because it only groups pixels by colour and doesn't actually understand what a "face" is. If the background has warm tones similar to skin, K-Means just merges the face with the background. Models like HOG + SVM and Haar Cascades were much better at ignoring this because they look for geometric patterns. However, I did discover that detectors trained on full silhouettes, like the default HOG-people detector, can fail if the person's legs are cut off in the photo.

Ultimately, MTCNN was the most accurate model by far. It successfully found my face and even detected a secondary person in that crowded Times Square image. This really demonstrates how deep learning models learn features in layers rather than just relying

on gradients that a human has defined. While classical methods like Haar Cascades were definitely fast, they produced way more false positives in chaotic settings compared to the three-stage refinement process in MTCNN. For future improvements, adding more pre-processing like Histogram Equalization could help with detection in low-light areas. Overall, this whole comparison proves that deep learning is the most robust choice for analyzing faces in the real world.

References:

- Lin, W. (2018). *Digital Image Processing in C++ Chapter 9: Thresholding, Roberts, Prewitt, Sobel and Edge*. [online] Medium. Available at: <https://medium.com/@wilson.linzhe/digital-image-processing-in-c-chapter-9-thresholding-roberts-prewitt-sobel-and-edge-e7428405ede3> [Accessed 19 Apr. 2026].
- Fisher, R., Perkins, S., Walker, A. and Wolfart, E. (2003). *Edge Detection*. [online] Hypermedia Image Processing Reference (HIPR2). Available at: <https://homepages.inf.ed.ac.uk/rbf/HIPR2/edgdetct.htm> [Accessed 19 Apr. 2026].
- Gunawardana, K. (2019) *Edge Detection with Laplacian Operator without using OpenCV Laplacian inbuilt function*. [Online] Medium. Available at: <https://kalpanileo1996.medium.com/edge-detection-with-laplacian-operator-without-using-opencv-laplacian-inbuilt-function-fa6d4966065f> [Accessed: 19 April 2026].